

04-grafana-dashboards-and-alerts

Grafana Dashboards and Alerts

1. Overview
2. Core Concepts
 - 2.1 Data Sources
 - 2.2 Dashboards
 - 2.3 Panels
 - 2.4 Variables (Templating)
 - 2.5 Annotations
3. Building a Dashboard — Best Practices
4. Grafana Alerting
 - 4.1 Alert Rule Components
 - 4.2 Notification Policies & Contact Points
 - 4.3 Alert States
5. Grafana + Prometheus: Two Alerting Options
6. Dashboards as Code
7. Summary
- Lab: Building a Dashboard and Alert in Grafana
 - Prerequisites
 - Step 1 — docker-compose.yml
 - Step 2 — prometheus.yml
 - Step 3 — Provision the Prometheus data source
 - Step 4 — Launch
 - Step 5 — Build a dashboard
 - Step 6 — Create a Grafana-managed alert
 - Step 7 — Trigger the alert
 - Step 8 — Cleanup
 - Reflection

Grafana Dashboards and Alerts

1. Overview

Grafana is an open-source visualization and analytics platform used to query, visualize, alert on, and explore metrics, logs, and traces regardless of where they're stored. It does not store data itself (aside from its own internal config/alerting DB) — it's a **query and visualization layer** on top of data sources like Prometheus, Loki, Elasticsearch, InfluxDB, CloudWatch, and many others.

2. Core Concepts

2.1 Data Sources

A data source is a connection to a backend that Grafana queries. Each data source has its own query language: - Prometheus → PromQL - Loki → LogQL - Elasticsearch → Lucene/DSL - InfluxDB → Flux/InfluxQL - CloudWatch → CloudWatch metrics API

Data sources are configured once (via UI or provisioning YAML) and then reused across many dashboards.

2.2 Dashboards

A dashboard is a collection of **panels** arranged in a grid layout, typically representing a coherent view of a system (e.g., “API Service Overview” or “Node Health”).

2.3 Panels

The visual building block of a dashboard. Common panel types: | Panel Type | Use Case | |—|—| | Time series | Trends over time (CPU, request rate, latency) | | Stat | Single current value (e.g., current error rate) | | Gauge | Value against a range/threshold (e.g., disk usage %) | | Bar gauge / Bar chart | Comparing categories | | Table | Tabular raw data, top-N lists | | Heatmap | Distribution over time (e.g., latency histogram) | | Logs panel | Raw log lines (via Loki/Elasticsearch) | | Node graph / Traces panel | Service maps and distributed traces |

2.4 Variables (Templating)

Variables let a single dashboard become reusable across environments/instances without duplicating it. For example, a `$namespace` variable lets a user pick a Kubernetes namespace from a dropdown, and every panel's query updates accordingly:

```
sum(rate(http_requests_total{namespace="$namespace"}[5m]))
```

Variable types: Query (populated by a data source query), Custom (static list), Interval (time step), Data source (switch entire data source), Constant.

2.5 Annotations

Vertical markers on time series panels denoting events (deployments, incidents). Can be manual, or automatically populated from a data source query (e.g., mark every Kubernetes deployment rollout).

3. Building a Dashboard — Best Practices

1. **Top-down layout** — most important/aggregate metrics at the top, drill-down detail below
2. **Consistent time range and refresh interval** across panels on the same dashboard
3. **Use the RED method for services** — Rate, Errors, Duration
4. **Use the USE method for resources** — Utilization, Saturation, Errors
5. **Avoid overcrowding** — a dashboard with 40 panels is unreadable; split into multiple focused dashboards linked together
6. **Use consistent color/units** — e.g., always red = errors, and always express latency in ms consistently
7. **Version dashboards as code** — store dashboard JSON in git and provision via config, rather than editing solely through the UI, so changes are reviewable and reproducible

4. Grafana Alerting

Grafana has its own unified alerting engine (Grafana-managed alerts), which can evaluate queries directly — separate from (or in addition to) Prometheus's own Alertmanager-based alerting.

4.1 Alert Rule Components

- **Query** — the data to evaluate (e.g., a PromQL expression)
- **Condition** — the threshold expression (e.g., `IS ABOVE 0.05`)
- **Evaluation interval** — how often the rule is checked (e.g., every 1m)
- **Pending period ("for")** — how long the condition must hold before firing, to avoid flapping
- **Labels/Annotations** — metadata attached to the fired alert, used for routing and readability

Example rule expressed conceptually:

```
Query: sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m]))
Condition: IS ABOVE 0.05
For: 5m
Labels: severity=critical, team=checkout
```

4.2 Notification Policies & Contact Points

- **Contact points** define *where* an alert goes: Slack, PagerDuty, Email, Webhook, OpsGenie, Microsoft Teams, etc.

- **Notification policies** form a tree that routes alerts to contact points based on their labels (similar in spirit to Prometheus Alertmanager's routing tree)
- **Silences** temporarily mute alerts matching a label set (e.g., during planned maintenance)
- **Mute timings** suppress notifications during recurring time windows (e.g., non-business hours for non-critical alerts)

4.3 Alert States

State	Meaning
Normal	Condition not met
Pending	Condition met, but “for” duration hasn’t elapsed yet
Alerting	Condition met and sustained — notification fires
No Data	The query returned no data (often itself worth alerting on)
Error	The query itself failed to execute

5. Grafana + Prometheus: Two Alerting Options

You can define alerts in **two** places, and it's important to pick deliberately:

1. **Prometheus PrometheusRule / Alertmanager** — alerts evaluated by Prometheus itself, routed via Alertmanager. Good when you already have a mature Prometheus/Alertmanager setup and want alerting to survive even if Grafana is down.
2. **Grafana-managed alerts** — alerts evaluated by Grafana against any data source (not just Prometheus), useful when mixing data sources (e.g., alerting on a Loki log-based query) or when you want a single UI for alerts across everything.

Many teams use Prometheus Alertmanager for infrastructure-critical alerts and Grafana-managed alerting for broader, cross-data-source dashboards.

6. Dashboards as Code

Grafana supports **provisioning**: dashboards and data sources defined as YAML/JSON files loaded automatically at startup, ideal for GitOps workflows.

```
apiVersion: 1
providers:
- name: 'default'
  folder: 'Kubernetes'
  type: file
  options:
    path: /etc/grafana/provisioning/dashboards
```

Tools like **Grafonnet** (Jsonnet library) or the **Grafana Terraform provider** are commonly used to generate/manage dashboard JSON programmatically at scale.

7. Summary

- Grafana visualizes data from many sources; it doesn't store metrics itself
- Dashboards = panels + variables + annotations, ideally following RED/USE method conventions
- Grafana-managed alerting is data-source-agnostic and complements Prometheus/Alertmanager alerting
- Notification policies + contact points route alerts to the right people/tools
- Treat dashboards and alerts as code — version them, review them, and provision them automatically

Lab: Building a Dashboard and Alert in Grafana

Goal: Connect Grafana to Prometheus, build a dashboard with a templated variable, and configure a firing alert routed to a webhook.

Prerequisites

- Docker & Docker Compose
- (Optional) Completion of the Prometheus architecture lab, or a running Prometheus instance with `node_exporter` scraped

Step 1 — docker-compose.yml

```
version: "3.8"
services:
  node_exporter:
    image: prom/node-exporter
    ports:
      - "9100:9100"

  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - ./provisioning:/etc/grafana/provisioning
```

Step 2 — prometheus.yml

```
global:
  scrape_interval: 10s
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node_exporter:9100']
```

Step 3 — Provision the Prometheus data source

```
mkdir -p provisioning/datasources
cat > provisioning/datasources/prometheus.yml <<'EOF'
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
EOF
```

Step 4 — Launch

```
docker compose up -d
```

Log into Grafana at `http://localhost:3000` (user: `admin`, password: `admin`).

Step 5 — Build a dashboard

1. Go to **Dashboards** → **New** → **New Dashboard** → **Add visualization**
2. Select the **Prometheus** data source
3. Enter query:

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

4. Set panel title to “CPU Utilization %”, visualization type “Time series”
5. Add a second panel for memory:

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

6. Add a **Variable**: Settings → Variables → New → Query type → data source Prometheus → query `label_values(instance)`, name it `instance`. Update both panel queries to filter `{instance="$instance"}`.
7. Save the dashboard as “Node Health Overview”

Step 6 — Create a Grafana-managed alert

1. Go to **Alerting → Alert rules → New alert rule**
2. Query: reuse the CPU utilization query from Step 5
3. Condition: `IS ABOVE 80`
4. Evaluation group: every `10s`, for `30s`
5. Add label `severity=warning`
6. Under **Contact points**, create a new webhook contact point pointing to a test endpoint (e.g., `https://webhook.site/your-unique-url` for testing)
7. Save and confirm the rule appears under Alert rules with state `Normal`

Step 7 — Trigger the alert

Generate CPU load on the host running `node_exporter` (e.g., `yes > /dev/null &` a few times), and watch the alert state transition **Normal → Pending → Firing** in the Grafana Alerting UI. Confirm the webhook receives the notification payload.

Step 8 — Cleanup

```
docker compose down -v
```

Reflection

1. Why might you choose Grafana-managed alerting over Prometheus Alertmanager for this specific rule?
2. What happens to your dashboard’s panels when you change the `$instance` variable — and why is this more maintainable than duplicating dashboards per host?
3. How would you extend the notification policy so `severity=critical` alerts go to PagerDuty while `severity=warning` alerts go to Slack only?